This version of the article has been accepted for publication, after peer review (when applicable) and is subject to Springer Nature's AM terms of use (https://www.springernature.com/gp/open-research/policies/accepted-manuscript-terms), but is not the Version of Record and does not reflect post-acceptance improvements, or any corrections. The Version of Record is available online at: https://doi.org/10.1007/s10479-024-05995-6

An open-source framework for solving shop scheduling problems in manufacturing environments

Carlos R. H. Márquez[1], Vanessa Braganholo[1], Celso Ribeiro[1*]

[1]Institute of Computing, Universidade Federal Fluminense, Brazil.

*Corresponding author(s). E-mail(s): celso@ic.uff.br;
Contributing authors: cherreram2012@gmail.com; vanessa@ic.uff.br;

Abstract

Scheduling problems refer to the decision-making process of allocating tasks to resources, usually scarce and in high demand, to optimize different performance measures. We consider the class of shop scheduling problems arising in the context of manufacturing systems, which are often NP-hard and challenging to solve. Exact methods have limitations in finding optimal solutions in reasonable computation times, even for instances of moderate size. Therefore, in real-life production environments, finding high-quality solutions is often satisfactory, even if they are not optimal. We contribute to the solution of shop scheduling problems with the design and implementation of the SSP-3M framework, oriented by three main guidelines: versatility, extensibility, and independence of the optimization method. These characteristics reduce the gap between scheduling theory and practice in real-life environments and improve the integration of the scheduling framework with other process planning or functions such as Computer-aided Process Planning, Advanced Planning and Scheduling, Integrated Process Planning and Scheduling, and Computer Integrated Manufacturing. The problem and solution representations adopted in our framework design make it possible to handle six shop scheduling problem variants, illustrating its versatility: job shop, flow shop, permutation flow shop, generalized flow shop, flexible flow shop, and flexible job shop. SSP-3M is open-source and can be used by any interested party. Our experimental evaluation shows that it can successfully be integrated with external optimization methods. We claim that SSP-3M is a good choice for companies that need free and quick-to-develop solutions to shop scheduling problems.

Keywords: Scheduling framework, shop scheduling, manufacturing systems, job shop problem, flow shop problem, scheduling algorithms, combinatorial optimization, open-source, manufacturing environment

1 Introduction

Scheduling problems refer to the decision-making process of allocating tasks to resources, usually scarce and in high demand, to optimize different performance measures, such as cost reduction, increased throughput, or production flexibility. In the context of manufacturing systems, Brucker [1] and other authors name the scheduling problems arising in manufacturing environments by *shop scheduling problems* (SSP) [2, 3]. In such problems, we are given a set of jobs $J = \{J_1, \ldots, J_n\}$ and a set of machines $M = \{M_1, \ldots, M_m\}$ where the jobs must be processed. Each job $J_i \in J$ consists of a set of m operations $O_i = \{o_{ij} : j = 1, \ldots, m\}$ that must be processed in a given order by every machine in M. Each operation o_{ij} has associated real values s_{ij}, p_{ij}, and c_{ij} that represent their start, processing, and completion times, respectively. There are precedence relations between the operations associated with each job (*sequence constraints*, also known as *technological route*). Each machine can process only one job simultaneously (*capacity constraint*). At any time, any job can not be under processing by multiple machines (*integrity constraint*). The processing times of each job on each machine are known, and the objective is to find a feasible schedule that optimizes some criterion. A feasible solution is characterized by starting and ending times for all operations, meeting the sequence, capacity, and integrity constraints.

Considering the workflow of each job and the machine environment, the class of shop scheduling problems includes the variants described below [4]. The little diagrams in Figure 1 illustrate each of the problem variants using four machines and three jobs. Machines (T, D, G, and M) are represented by little colored pieces, and jobs (#1, #2, and #3) are represented by dotted colored arrows.

(i) *Flow shop*: There are m machines in series. Each job has to be processed on each of them. All jobs follow the same route, i.e., their processing starts on the first machine, then on the second, and so on until the last machine is used. Each job completed on a given machine joins the queue at the next machine. In the diagram corresponding to the flow shop in Figure 1, each job is represented in the queue for each machine to show that each of them is processed on every machine. The order in which the jobs are processed on each machine may vary. For example, in the figure, job #1 is processed first in machine T but last in machine D.

(ii) *Permutation flow shop*: This is a particular case of the flow shop, where all jobs must be processed in the same order in every machine. The queues formed in each machine operate under the first-in, first-out discipline. Note how, in the diagram corresponding to the permutation flow shop in Figure 1, the jobs appear in the same order on each machine.

(iii) *Generalized flow shop*: In this generalization of the flow shop, the jobs do not have to be processed in all machines. In the diagram of Figure 1, job #1 is not processed in machine D, and job #3 is not processed in machine G.

(iv) *Flexible flow shop*: This is a variant of the flow shop on parallel machine environments. Instead of m machines in series, there are some stages in series and some parallel machines at each stage. Each stage behaves as a bank of parallel machines. All jobs follow the same route between the stages. In the illustration provided by

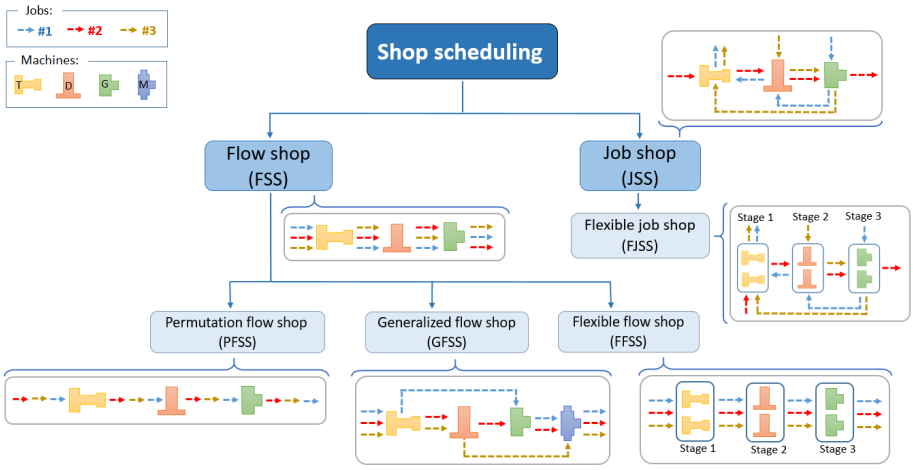


Fig. 1 Hierarchy of shop scheduling problem variants based on the job workflow.

Figure 1, the parallel banks of machines are represented by machines inside a blue rounded rectangle.

(v) *Job shop*: Each job has its predetermined route to follow. A route is a sequence of machines used by a given job. If *recirculation* is allowed, a job may visit a machine more than once. In the figure, this is represented by jobs using machines in very different orders.

(vi) *Flexible job shop*: This is a variant of the job shop on parallel machine environments. There are some work centers and some parallel machines in each work center. Each job has its predetermined route to follow between the work centers.

Shop scheduling problems appropriately describe the characteristics of jobs, machine tools, cutting tools, machine setup times, cutting tool paths, and technological routes that appear in machining industries and almost all manufacturing environments, independently of the industry type (such as electronics, aerospace, automotive, food, pharmaceutics, textile, mining, agriculture, energy, and shipping), of the production process (such as casting, metal forming, machining, welding, and 3D printing), of the production type (unit, serial, massive), or of the raw materials (such as polymers, metals, ceramic, felt paper, and glass) [5].

These problems are often NP-hard and challenging to solve. Exact methods have limitations in finding optimal solutions in reasonable computation times, even for instances of moderate size. Therefore, in real-life production environments, finding high-quality solutions is often satisfactory, even if they are not optimal. Although more advanced research focuses on complex exact algorithms to address problems that are often abstractions of reality, the industry faces more day-to-day practical needs such as the integration of the scheduling with Computer-aided Process Planning (CAPP), Advanced Planning and Scheduling (ASP), Integrated Process Planning and Scheduling (IPPS), and Computer Integrated Manufacturing (CIM) [3], or algorithms capable of tackling constraints that appear in fact in the shop floor, such as the transportation

times from one machine to the other, job recirculation, machine availabilities, machine flexibility, changes in task priorities or batches of tasks.

Several authors mention a persisting a gap between the theory and practice in solving the shop scheduling problem [6–8]. Márquez and Ribeiro [3] noticed an increase in the interest in the integration of SSP solutions with other process planning functions or systems such as CAPP, APS, IPPS, or CIM, which justifies the development of a framework to provide this integration.

The main contribution of this work is the design, implementation, and test of a framework for solving the many variants of static shop scheduling problems (and not a single, specific variant), taking into account characteristics of real-life production environments, such as machine flexibility, task priorities, due dates, and energy consumption. This framework should be versatile, extensible, independent of the optimization methods, and integrated with other process planning functions. The SSP-3M framework proposed in this work handles six shop scheduling problem variants (job shop, flow shop, permutation flow shop, generalized flow shop, flexible flow shop, and flexible job shop) with eight performance metrics (makespan, flowtime, idle time, lateness, tardiness, number of early and tardy jobs, and energy-consumption) that may be used as objective functions, characterizing the versatility that was one of its design goals.

The framework takes its acronym from the fact that it handles multiple problem variants, objective functions, and optimization methods, in addition to its three design features: versatility, extensibility, and independence from the optimization method. SSP-3M is open-source, ready-to-use, and can be used by any interested party.

This article is organized as follows. The next section presents the literature review and describes previous proposals and efforts to develop frameworks or architectures for SSP. Section 3 describes in detail the components of the proposed framework: (i) the data layer, (ii) the problem-specific layer, and (iii) the visualization layer. Section 4 shows some numerical results obtained from the straightforward integration of the framework with two externally developed metaheuristics: a genetic algorithm (a population-based metaheuristic) and a GRASP (a trajectory-based metaheuristic), on a benchmark of well-known instances of the flow shop, generalized flow shop, job shop, and flexible job shop problems. Finally, Section 5 presents the main conclusions and managerial implications of this work.

2 Architectures for scheduling problems

We have seen in Section 1 that there is a gap between the theory and practice concerning solving shop scheduling problems in real life [3, 6, 9, 10]. One of this work's goals is to design and implement a framework for solving shop scheduling problems (SSP) to be integrated with process planning functions or systems such as CAPP, APS, IPPS, or CIM.

This section reviews some efforts to develop frameworks and software architectures for solving shop scheduling problems, with their characteristics and limitations. Some of these frameworks have been implemented, while others are just conceptual

or proposals. King [6] and Buxey [9] were the first to identify the gap between theory and practice in scheduling environments. Reisman et al. [10] showed that only a tiny fraction of the publications in scheduling addressed the actual characteristics of real-life manufacturing systems. Márquez and Ribeiro [3] showed that this situation has mostly stayed the same in the last 20 years, and this gap has increased with the new challenges. According to Pongcharoen et al. [11], although static and deterministic scheduling problems are easy to formulate, they are hard to solve due to their combinatorial nature. However, this hardship gets even higher for practical cases or when the characteristics of real-life environments are contemplated.

Reducing this gap has motivated several works that focus on creating frameworks to support the solution of shop scheduling problems in real-life production environments. Framinan and Ruiz [12] reviewed the literature focusing on the characteristics, strategies, and recommendations that should be considered when developing a scheduling system, particularly data exchange, solution procedures, objective functions and constraints, modular design, and implementation.

Some approaches in the literature rely on specific optimization methods, such as those of Sun and Lin [13], Lee et al. [14], Vieira et al. [15], Lau and Qiu [16], Pongcharoen et al. [11], Fumagalli et al. [17], and Liu et al. [18]. Sun and Lin [13] developed a dynamic scheduling framework for solving the job shop problem using mixed integer linear programming. It facilitates the integration with process planning functions by establishing the relationship between scheduling, due date assignment, and job release times through backward scheduling. The basis of this framework is the so-called "backward scheduling", a procedure working with reversed routing for each job. The dynamic scheduling problem is solved through a series of backward scheduling subproblems in which the jobs to be scheduled and their release times are determined dynamically. The other approches are based on metaheuristics, particularly genetic algorithms. Lee et al. [14] developed a framework for a job shop scheduling system combining a genetic algorithm with induced decision trees, a machine learning technique. Three main components integrate it: (i) an inductive learning technique for job release, (ii) a genetic algorithm for dispatching in the available machine, and (iii) a shop floor current state updater. The framework was tested on computer simulation experiments for two case studies. Vieira et al. [15] presented a conceptual framework for rescheduling manufacturing systems, which includes defining the elements in almost all rescheduling manufacturing systems: rescheduling strategies, environments, policies, and methods. Lau and Qiu [16] presented a unified framework for solving generic job shop scheduling problems based on artificial immune systems. It consists of four modules: (i) job shop database, (ii) algorithm execution, (iii) job shop scheduling processing and controlling, and (iv) job shop evaluation module. It should be able to handle static, semi-dynamic, and dynamic problems. A high-level decision support model allows the application of the framework to solve real job shop problems. Fumagalli et al. [17] described a scheduling framework combining a genetic algorithm with an optimization algorithm and a simulation model integrated with the evaluation of the objective function to handle the stochasticity of the production system. The proposal was implemented on a real-life industrial case for a company that produces forged and laminated rolled rings of several materials, from carbon and alloyed steels

to nickel, titanium, cobalt alloys, aluminum, and copper. Liu et al. [18] proposed a hybrid manufacturing/remanufacturing system with demand substitution for solving scheduling in stochastic environments. Their non-modular proposal is formed by an ant colony algorithm with a random sampling method that is used to minimize the total expected cost.

Contrarily, the works of Yen and Pinedo [19], Pinedo and Yen [20], Bidot et al. [21], Framinan and Ruiz [12, 22], and Romero-Silva et al. [23] provide independence of the optimization method. Yen and Pinedo [19] tackled several issues related to the design and development of scheduling systems for manufacturing, considering three common aspects: (i) the database and knowledge base modules, (ii) the scheduling engine modules, and (iii) the user interface modules. They also described possible architectures with their advantages and drawbacks. Pinedo and Yen [20] described the architecture of an object-oriented scheduling system formed by three main components: (i) the database, (ii) the schedule generator, and (iii) the user interface. The architecture is highly modular, so the resulting scheduling systems are easily reconfigurable and scalable. Bidot et al. [21] proposed a framework for solving scheduling in a stochastic environment. It considers proactive, progressive, and revision approaches for stochastic environments. The authors describe some software prototypes that could be instantiated from the proposed framework. Framinan and Ruiz [22] developed an integrated and modular architecture for realistic manufacturing scheduling systems. It is composed of five components or modules: (i) user interface, (ii) schedule generator, (iii) business logic and data abstraction, (iv) database management, and (v) business information system. Romero-Silva et al. [23] also proposed a conceptual framework for production scheduling environments. It has three main three modules: (i) production environment, (ii) activity monitoring, and (iii) schedule generation. However, we observe that most of such frameworks and software architectures handle only a unique problem variant and fail to support the use of more than two objective functions, while our work currently supports eight.

This overview of the literature about framework proposals and prototypes for shop scheduling problems shows different architectures whose features and characteristics are summarized in Table 1. Several of these approaches are not fully characterized since they are merely conceptual proposals or general implementation guidelines. To the best of our knowledge, ours is the only open-source solution to SSP. Additionally, most of them miss the characteristics of real-life environments, such as machine flexibility, task priorities, due dates, and energy consumption, which makes them less useful in practice. In fact, the framework proposed in this work is the only one to support all of these features. No other approach supports task priorities, due dates, and energy consumption, and only two other approaches [17, 21] are able to deal with machine flexibility.

Table 1 Features and characteristics of different frameworks and software architectures for shop scheduling problems (MILP: mixed-integer linear programming; GA: genetic algorithm; ML: machine learning; AIS: artificial immune systems; ACO: ant colony optimization; n.a.: information not available).

Article	Open source?	Approach implemented?	Optimization method independence?	Problem variants	User interface?	Objective functions	Numerical results?
Yen and Pinedo [19]	no	no	yes	n.a.	no	n.a.	no
Sun and Lin [13]	no	yes	no (MILP)	job shop	no	flow time, tardiness	no
Lee et al. [14]	no	yes	no (GA/ML)	job shop	no	makespan, tardiness	yes
Pinedo and Yen [20]	no	no	yes	n.a.	yes	n.a.	no
Vieira et al. [15]	no	no	no (GA)	n.a.	no	n.a.	no
Pongcharoen et al. [11]	no	yes	no (GA)	job shop	yes	earliness, tardiness	no
Bidot et al. [21]	no	yes	yes	flexible job shop	no	makespan, tardiness	yes
Framinan and Ruiz [22]	no	no	yes	n.a.	no	n.a.	no
Framinan and Ruiz [12]	no	no	yes	n.a.	no	n.a.	no
Lau and Qiu [16]	no	yes	no (AIS)	job shop	no	makespan	no
Romero-Silva et al. [23]	no	no	yes	n.a.	no	n.a.	no
Fumagalli et al. [17]	no	yes	no (GA)	flexible job shop	no	cost	yes
Liu et al. [18]	no	yes	no (ACO)	job shop	no	cost	yes
This work	yes	yes	yes	job shop, flow shop, permutation flow shop, generalized flow shop, flexible flow shop, flexible job shop	yes	makespan, flowtime, idletime earliness, tardiness, early jobs, tardy jobs, energy consumption	yes

The framework presented in the next section builds upon the conclusions drawn and the needs observed in the literature review. The problem representation using a directed disjunctive graph and the solution representation by sequences of jobs and operations make it possible to handle the six shop scheduling problem variants and illustrate the framework's versatility. The numerical simulations show that the framework is independent of the selected optimization methods. The object-oriented design ensures its extensibility, by which users of the framework may introduce additional components such as new heuristics for computing initial solutions, new metaheuristics, new performance criteria, and the representation of new problem constraints.

3 Architecture and components of the SSP-3M framework

Figure 2 shows the schematic representation of the SPP-3M framework and its main components. It contains three layers: (i) data layer, dedicated to manipulating and storing the input data; (ii) problem-specific layer: dedicated to describing the specific elements of the SSP; and (iii) visualization layer: dedicated to solution output and visualization, using Gantt and electric consumption charts. The boxes in grey represent the main components of the framework. The user can extend the grey boxes with red dotted lines (e.g., reading from different input sources, implementing new objective functions, or constructive heuristics, such as NEH [24, 25]), or integrate them with external components such as metaheuristics (represented by a blue box with blue dotted lines). We note that the metaheuristics are not part of the framework, i.e., the SSP-3M framework is independent of the metaheuristic and may be combined with any of them.

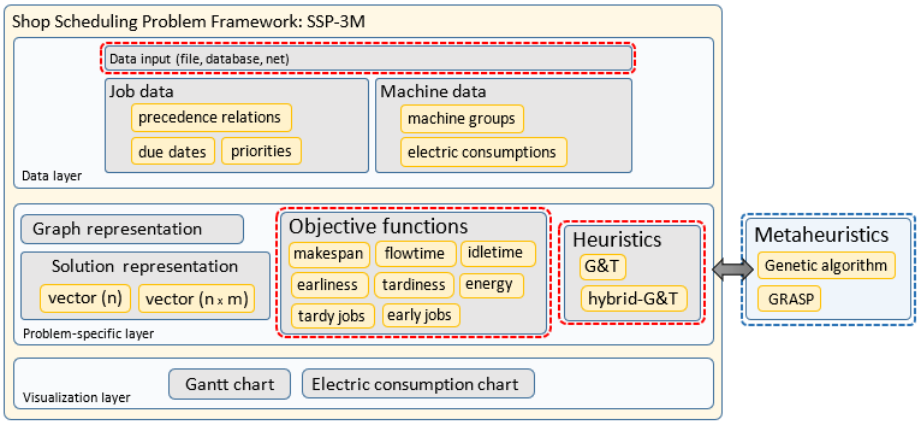


Fig. 2 Schematic representation of the proposed SSP-3M framework.

The choice of which framework component should be used depends on the specific problem that needs to be solved. The extensible features of the SSP-3M framework and the way it represents data internally allow the framework to handle the six shop

scheduling problem variants (job shop, flow shop, permutation flow shop, generalized flow shop, flexible flow shop, and flexible job shop) shown in Figure 1. In the following subsections, we detail each layer and explain how the framework is able to handle the different SSP variants.

3.1 Data layer

The data layer gets the input data and stores all job and machine data that define the capabilities and limitations of the problem representation and solution. The problem data contains the following elements, in this order: (i) numbers of jobs and machines; (ii) list of available machines; (iii) electrical rated power for each machine; and, for each job, (iv) its name, identification, priority, due date, penalty, prize, and the sequence of operations.

Figure 3(a) illustrates an example of the file structure the framework uses for data input. The first two lines inform the number of jobs and machines. The remainder of the file is organized into three sections. The first section (lines 4 to 8) is labeled `MACHINES` and declares the identifiers of each machine. The second section (lines 10 to 14) is labeled `RATED-POWER` and informs the nominal power consumption of each declared machine. Finally, there is a section labeled `JOB` for each job, containing the following attributes: the job name, identifier, priority, release date, due date, penalty, and prize. Zeroes replace irrelevant attributes. The description of each job concludes with its sequence of operations. Each operation is defined by a machine where it will be executed and its processing time. As an example, the first job represented in Figure 3 (lines 15 to 27) is called $J1$ (line 16) and has three operations. The first operation takes two time units to execute on machine $M1$ (line 24); the second takes three time units to execute on machine $M2$ (line 25); and the third one takes five time units to execute on machine $M3$ (line 26). This input file could be used in the job shop variant of the SSP problem since each job has its own route (that is, they use the machines in different orders). The diagram shown in Figure 3(b) illustrates this scenario.

The framework can be extended to add other forms of input data. Internally, data is represented as objects that are accessible to the problem-specific layer.

3.2 Problem-specific layer

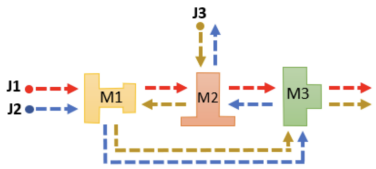
This section describes the framework components dedicated to representing the elements of the shop scheduling problem to be solved: (i) problem representation using a directed disjunctive graph, (ii) solution representation, (iii) constructive heuristics for creating initial solutions, and (iv) objective functions.

3.2.1 Problem representation

The problem representation is based on a directed disjunctive graph $G = (V, I^+ \cup C \cup D \cup E^-)$, where there is a node in V associated with each operation o_{ij}, for every $i = 1, \ldots, n$ and $j = 1, \ldots, m$, where n is the number of jobs and m is the number of machines [26]. The node set V has two additional nodes I and E representing the beginning and end of all operations. C is the set of *conjunctive arcs* representing the precedence relations between the operations. For every pair of operations u and v

```
1.Jobs     : 3
2.Machines: 3
3.
4.[MACHINES]
5.M1
6.M2
7.M3
8.[End]
9.
10.[RATED-POWER]
11.1
12.1
13.1
14.[End]
15.[JOB]
16.Name: J1
17.ID: 1
18.Priority: 0
19.Release : 0
20.DueDate : 0
21.Penalty : 0
22.Prize   : 0
23.
24.M1: 2.0
25.M2: 3.0
26.M3: 5.0
27.[End]
28.[JOB]
29.Name: J2
30.ID: 2
31.Priority: 0
32.Release : 0
33.DueDate : 0
34.Penalty : 0
35.Prize   : 0
36.
37.M1: 6.0
38.M3: 5.0
39.M2: 3.0
40.[End]
41.[JOB]
42.Name: J3
43.ID: 3
44.Priority: 0
45.Release : 0
46.DueDate : 0
47.Penalty : 0
48.Prize   : 0
49.
50.M2: 4.0
51.M1: 3.0
52.M3: 4.0
53.[End]
```

(a) Input file



(b) Diagram

Fig. 3 (a) Typical structure of an input file for a problem with three jobs and three machines. Line numbers, shown in blue, are not part of the file. Portions of the file are shown side by side for conciseness. (b) Diagram representing the job workflow for the input data file.

using the same machine, there are two *disjunctive arcs* (u, v) and (v, u) [27]. A weight p_{uv} associated with the arc (u, v) represents the processing time of operation u. The arc subset I^+ is formed by arcs of null weight connecting the initial node I to the first operation of every job $J_i, i = 1, \ldots, n$. Finally, the arc subset E^- is formed by arcs connecting the last operation of every job $J_i, i = 1, \ldots, n$, to the final node E, with weights equal to their processing times.

Figure 4 illustrates the directed disjunctive graph associated with the instance with three jobs J_1, J_2, J_3 and three machines M_1, M_2, M_3 whose input file appeared in Figure 3. Solid lines represent the conjunctive arcs (set C), while dashed lines represent the disjunctive arcs (set D).

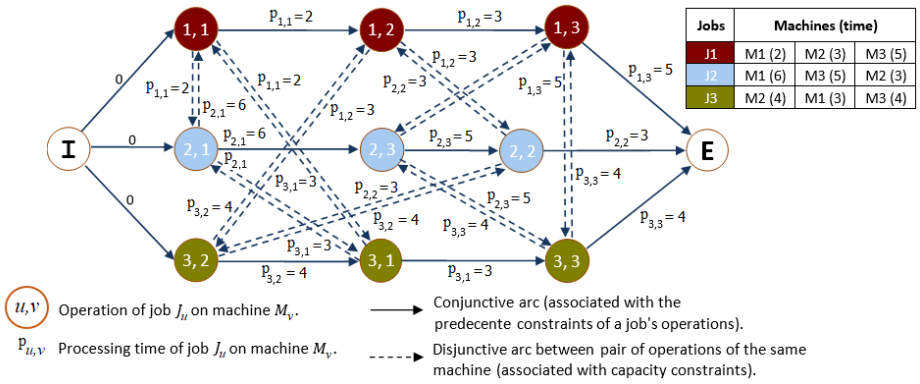


Fig. 4 Disjunctive graph associated with the instance with three jobs and three machines whose input file appeared in Figure 3.

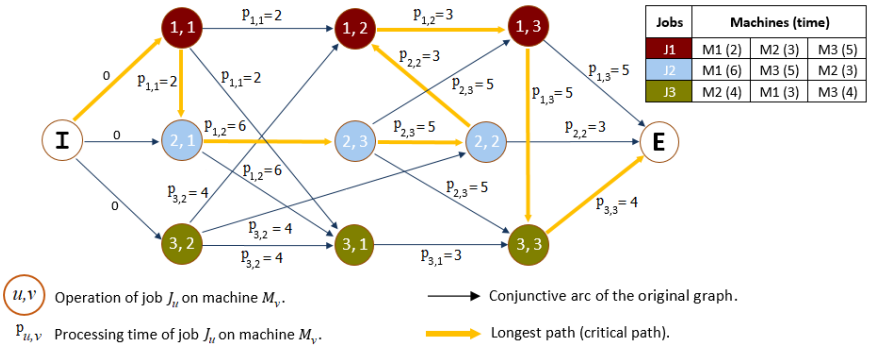


Fig. 5 Directed graph corresponding to a feasible schedule for the problem represented in Figure 4 (arcs of the critical path are shown in yellow).

Solving the scheduling problem using this representation consists of sequencing all operations that must be processed on each machine. It amounts to eliminating one of the arcs in each pair of disjunctive arcs and fixing the other, which becomes a conjunctive arc. The set of arcs resulting from this process is called a *selection* [26]. A selection S defines a *feasible schedule* if, and only if, (i) an arc is selected from every pair of disjunctive arcs, and (ii) the graph $G(S) = (V, I^+ \cup C \cup S \cup E^-)$ is acyclic. In this case, S is a *complete selection*.

Given a *schedule* associated with a complete selection S, the completion time of all jobs is the *makespan*, which is the weight C_{max} of the longest (or critical) path from node I to E in the graph $G(S) = (V, I^+ \cup C \cup S \cup E^-)$. Figure 5 illustrates the acyclic graph resulting from a complete selection associated with the example presented in Figure 4. The critical path appears in yellow.

3.2.2 Solution representation

Flexible solution representations facilitate the implementation of solution algorithms and improve the performance of the search space exploration. We describe three solution representations implemented in the SSP-3M framework:

(i) Job sequence

Each solution is a permutation of the jobs $J_1, \ldots, J_n$. This representation is appropriate for the permutation flow shop, where every machine processes all jobs in the same order. Considering the example of Figure 4 with three jobs and three machines, some feasible solutions are represented by (J_2, J_3, J_1) and (J_1, J_3, J_2). There are $n!$ solutions using this representation, and each solution is represented by a vector of size n [28].

(ii) Operation sequence without repetitions

Each solution is represented by a vector whose components are jobs, organized in m sequential partitions. The j-th partition is formed by the j-th operations of the jobs in this partition, for $j = 1, \ldots, n$. This partition contains a permutation of the

jobs $J_1, \ldots, J_n$, indicating the order in which they are processed, with no repetitions allowed. This representation allows different job processing sequences for each machine. It is appropriate for the following SSP variants flow shop and flexible flow shop. Considering the example of Figure 4, some feasible solutions are represented by $(J_2, J_3, J_1 \,|\, J_3, J_1, J_2 \,|\, J_2, J_3, J_1)$ and $(J_1, J_2, J_3 \,|\, J_1, J_3, J_2 \,|\, J_3, J_1, J_2)$. Since each of the m machines can go through a different job processing order, there are $n!^m$ solutions using this representation and each solution is represented by a vector of size $n \times m$ [29].

(iii) Operation sequence with repetitions

Each solution is represented by a vector whose components are jobs, organized in m sequential partitions. Each partition still contains a list of n jobs, but in this case, repetitions are allowed. This representation is appropriate to handle the variants job shop, flexible job shop, and generalized flow shop. Considering the example of Figure 4, some feasible solutions are represented by $(J_1, J_1, J_3 \,|\, J_3, J_1, J_2 \,|\, J_2, J_3, J_2)$ and $(J_3, J_2, J_1 \,|\, J_3, J_3, J_1 \,|\, J_2, J_1, J_2)$. Since the jobs can recirculate in the same machine, there are $(nm)!/(m!^n)$ solutions using this representation and each solution is represented by a vector of size $n \times m$ [30].

3.2.3 Construction heuristics

We describe the two most used construction heuristics for the job shop and flexible job shop problems. Their implementations offered in the SSP-3M framework use the *operation sequence with repetitions* solution representation. We use Gantt charts to graphically illustrate the scheduling of jobs to machines along the time, indicating the time interval where each job is processed on each machine. Three types of feasible SSP schedules are relevant in this context: semi-active, active, and non-delay.

(i) Semi-active schedules

The processing of at least one job in one machine can be anticipated without delaying any other job. Figure 6 illustrates the Gantt chart of a typical semi-active schedule.

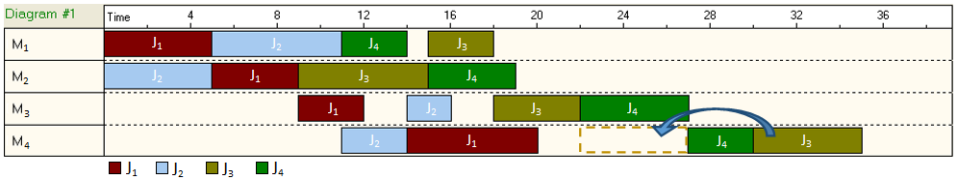


Fig. 6 Gantt chart of a semi-active schedule: job J_3 can be advanced and processed before job J_4 in machine M_4, without delaying J_4.

(ii) Active schedules

The processing of no job in any machine can be anticipated without delaying another job. Figure 7 illustrates the Gantt chart of a typical active schedule. Every active

schedule is semi-active [4]. Giffler and Thompson [31] proved that any schedule minimizing the makespan is active. Therefore, the search for a schedule minimizing the makespan may be limited to active schedules. Their greedy randomized G&T algorithm with time complexity $O(mn)$, described in detail as Algorithm 1 in Appendix A, is the most used algorithm for creating active schedules and will also be explored here.

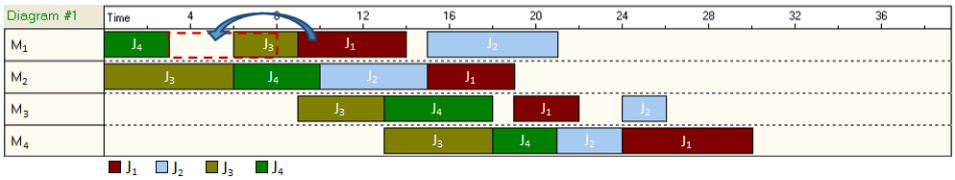
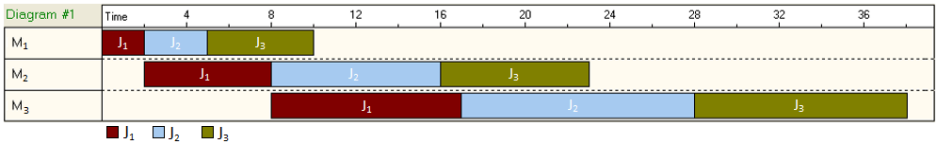


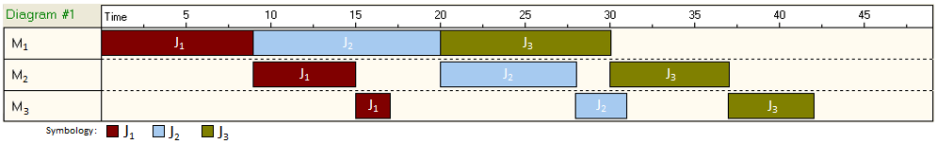
Fig. 7 Gantt chart of an active schedule: if a job such as J_1 is advanced and processed before job J_3 in machine M_1, job J_3 will be delayed.

(iii) Non-delay schedules

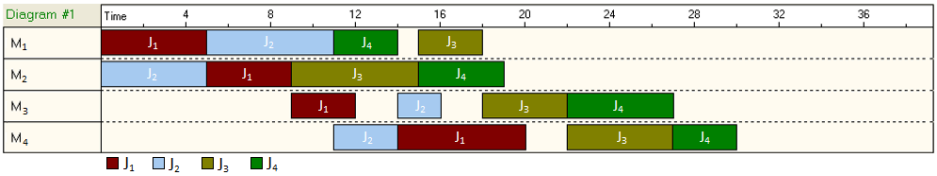
There is no free machine at the same time any job is ready to be executed on this machine. In other words, there is no idle time to anticipate any job. A non-delay schedule can be in one of the three situations: (i) there is no free (idle) machine, (ii) there is no free (ready to be executed) job, and (iii) there is neither a free machine nor a free job. Figures 8(a) to 8(c) illustrate the Gantt charts of typical non-delay schedules. Every non-delay schedule is active [4].



(a) Non-delay schedule with no free machines



(b) Non-delay schedule with no free jobs



(c) Non-delay schedule with no free jobs and no free machines

Fig. 8 Gantt chart of non-delay schedules.

Bierwirth and Mattfeld [32] showed that some active schedules minimizing the makespan are also non-delay. They explored this property to speed up the search for schedules that minimize the makespan. Their hybrid G&T algorithm is a variant of Algorithm 1 capable of producing active non-delay schedules using an adjustable parameter $\delta \in [0, 1]$ that reduces the number of operations in set B at each iteration. The reduction is maximum for $\delta = 0$, when only non-delay schedules are produced. In contrast, there is no reduction for $\delta = 1$, and the algorithm behaves as the original G&T algorithm, producing only active schedules. The only modification occurs in line 10 of Algorithm 1, where set B is computed, which should be replaced by:

10(a): $B \leftarrow \{o_{ik} \in A : k = k'\}$
10(b): $o_{i'',k''} \leftarrow argmin_{o_{ik} \in B}\{s_{ik}\}$
10(c): $B \leftarrow \{o_{ik} \in B : s_{ik} < s_{i'',k''} + \delta((s_{i',k'} + p_{i',k'}) - s_{i'',k''})\}$

The classic and hybrid G&T algorithms are implemented in the current version of the SSP-3M framework to generate active and non-delay schedules. Section 4.1 illustrates some numerical results obtained with their application to some well-known instances of the flow shop problem [33].

3.2.4 Objective functions

Figure 9 illustrates the schematic representation of the execution of a job J_i along the time, indicating the primary data and variables relative to its execution. We notice that r_i is the release date of job J_i (i.e., the time it becomes available for execution) and d_i its due date (i.e., the latest time for its completion). Both of these values are read at the data layer. Variables S_i and C_i are the starting time and completion time of job J_i. $F_i = C_i - S_i$ is the flowtime of job J_i, i.e., the total time it is processed in the environment, from its starting time until completion. $L_i = C_i - d_i$ represents the lateness of job J_i: $L_i > 0$ if the job is late; $L_i \leq 0$ otherwise.

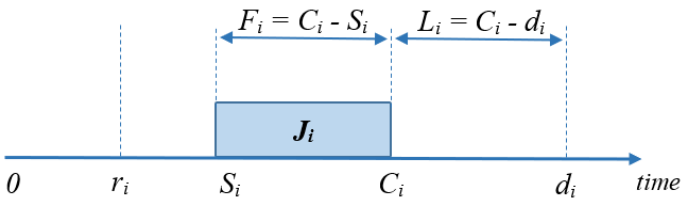


Fig. 9 Data and variables associated with the execution of a job.

Several performance metrics or objective functions exist for shop scheduling problems [1, 4, 34]. The following single objective performance metrics are implemented in the SSP-3M framework:

Metrics based on the processing times:

(i) Makespan: $C_{max} = max_{i=1,\ldots,n} C_i$, where C_i is the completion time of job J_i.
(ii) Total flowtime: $F_{total} = \sum_{i=1}^{n} F_i$, where F_i is the flowtime of job J_i.

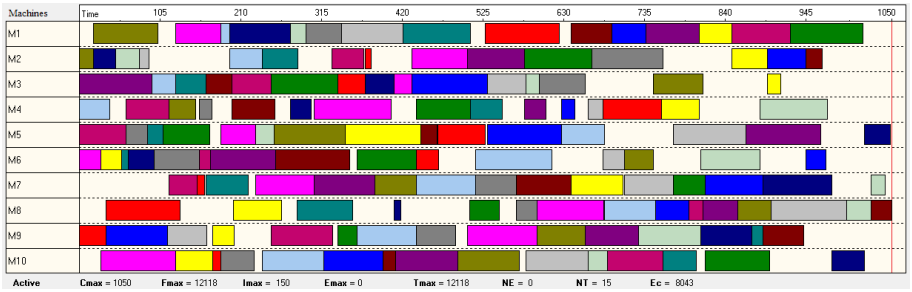


Fig. 10 Gantt chart for instance la21 with makespan equal to 1050.

(iii) Total idle time: $I_{total} = \sum_{j=1}^{m} I_j$, where I_j is the idle time of machine M_j. It corresponds to the total time this machine is available with at least one job ready to be executed.

Metrics based on the due dates:

(iv) Total lateness: $L_{total} = \sum_{i=1}^{n} L_i$, with $L_i = C_i - d_i$, where d_i is the due date of job J_i. The lateness L_i of job J_i is positive if J_i is late, i.e., $C_i > d_i$. The lateness L_i is negative if job J_i is completed before its due date, i.e., $C_i < d_i$.

(v) Total tardiness: $T_{total} = \sum_{i=1}^{n} T_i$, with $T_i = max\{L_i, 0\}$. The tardiness T_i of job J_i is always positive or null.

(vi) Number of early jobs: $Ne = \sum_{i=1}^{n} E_i$, where $E_i = 1$ if job J_i is completed before its due date, i.e., $C_i \leq d_i$; $E_i = 0$ otherwise.

(vii) Number of tardy jobs: $Nu = \sum_{i=1}^{n} U_i$, where $U_i = 1$ if job J_i is late, i.e., $C_i > d_i$; $U_i = 0$ otherwise.

Metric based on the energy use along the makespan, measured at k equally-spaced time intervals:

(viii) Energy consumption: $Ec = \sum_{t=1}^{k} \sum_{j=1}^{m} Y_{tj} P_i (C_{max}/k)$, where $Y_{tj} = 1$ if machine M_j is processing a job at the t-th time measurement; $Y_{tj} = 0$ otherwise. P_j is the rated electrical power of machine M_j and k is the total number of measurements along equal size intervals of the makespan.

3.3 Visualization layer

The visualization layer implements two widely used charts to represent SSP schedules: (i) Gantt charts and (ii) electric consumption charts.

Figure 10 illustrates the Gantt chart generated for instance la21 proposed in Lawrence [35], with 15 jobs and ten machines. Each row corresponds to a machine. Boxes with the same color correspond to operations of the same job.

The electric consumption chart displays graphically the electrical consumption for a given schedule. The vertical axis shows the power consumption at discretized time stamps displayed in the horizontal axis. Figure 11 illustrates the electrical consumption chart for the same instance la21 along the duration of 1050 time units corresponding to the makespan.

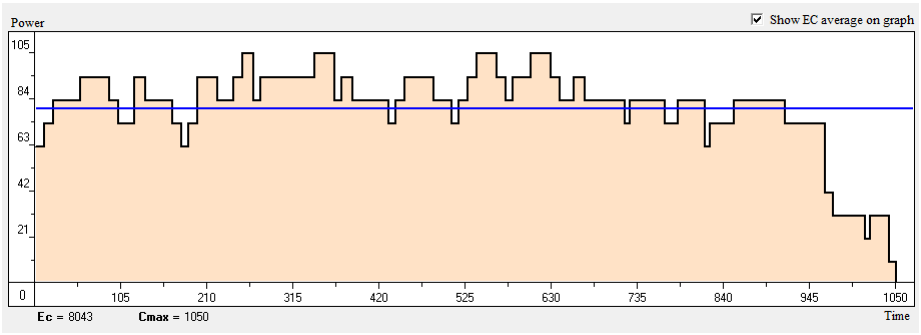


Fig. 11 Electrical consumption chart for instance la21 along the 1050 time units of the makespan.

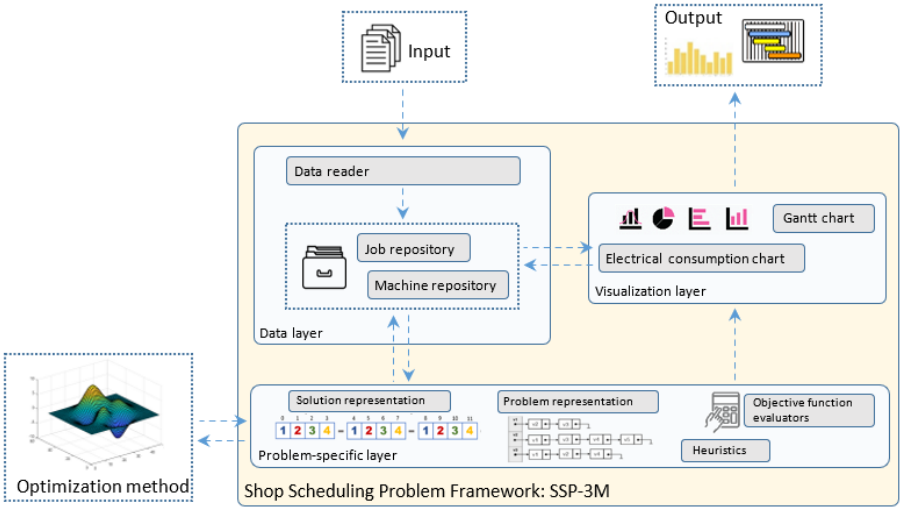


Fig. 12 High-level representation of the information flow on the SSP-3M framework.

3.4 Framework structure

Figure 12 shows a high-level representation of the proposed framework. The three layers of Figure 2 are shown in Figure 12 together with the flow information that follows inside the framework. Input data is read using the data reader component. From the input data, job and machine data are stored in the main memory in what we call `repositories`. The problem-specific layer uses these data to build the solution, which can later be displayed graphically by the components of the visualization layer.

The SSP-3M framework was built using object orientation. We use C++ to implement our framework. C++ does not support the concept of interfaces, which are usually the more elegant way of providing extensibility. As such, extending our framework is a matter of implementing one or more of its abstract classes or implementing a class that extends an existing concrete class and adding new methods to it.

Figure 13 shows a simplified class diagram of the framework. Classes marked in blue implement the data layer of the framework, while classes marked in red implement

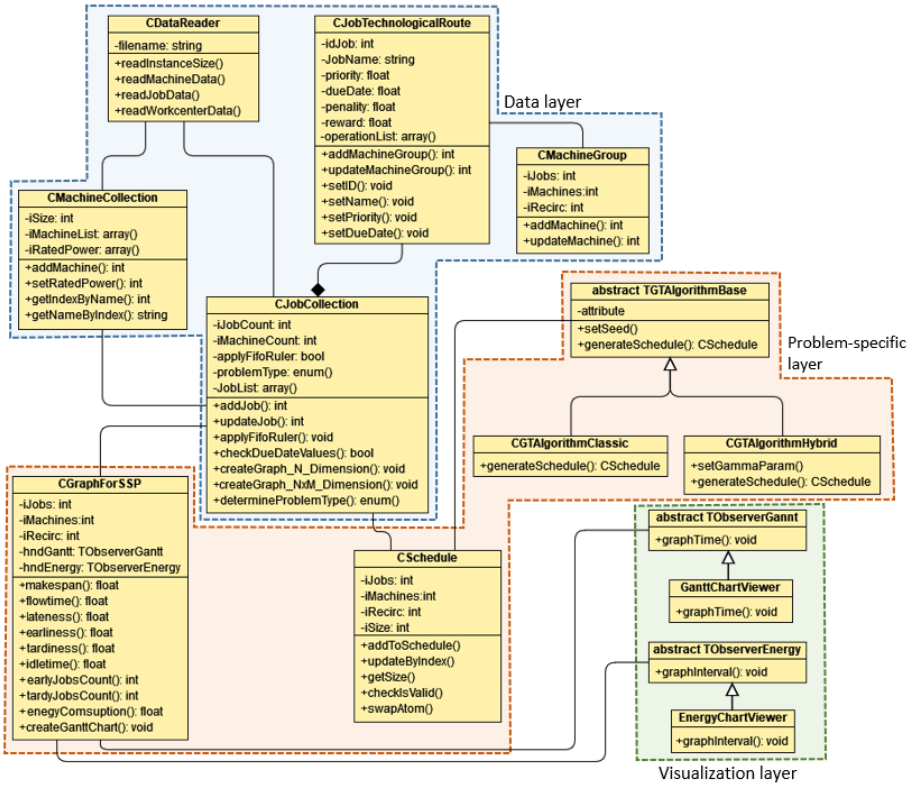


Fig. 13 UML diagram for the proposed framework.

the problem-specific layer, and classes marked in green implement the visualization layer. Classes with names starting with C are concrete classes, and classes with names starting with T are abstract classes. The complete and more complex UML class diagram is available at the project repository [36]. Below we provide a description of each of the classes of Figure 13.

- *CDataReader*: The *CDataReader* class implements methods that perform data input. This class can be extended to use other data sources, like JSON and XML files, database access, or data input through network services.
- *CMachineCollection*: The *CMachineCollection* class represents a collection of machines. It works as a repository that stores information about the machines.
- *CMachineGroup*: The *CMachineGroup* class represents the concept of Group Technology and flexible environments with machines in parallel. Márquez and Ribeiro [3] describe how scheduling integrates with Group Technology and flexible environments with machines in parallel.
- *CJobTechnologicalRoute*: This class represents data about jobs, including the sequence of operations of the job (technological route) as well as the processing times associated with each operation. The sequence of operations is stored in the *operationList* attribute of this class.

- *CJobCollection*: The *CJobCollection* class represents a collection of jobs.
- *CGraphForSSP*: This class implements the directed disjunctive graph that represents the SSP problem to be solved (see Section 3.2.1). The *CGGraphForSSP* class can be extended by the framework user to include new objective functions.
- *CSchedule*: The *CSchedule* class represents the solution to the scheduling problem, represented as an operation sequence of size n or $n \times m$, as discussed in Section 3.2.2.
- *CGTAlgorithmBase, CGTAlgorithmClassic, CGTAlgorithmHybrid*: These classes implement two constructive algorithms that are used to generate the initial solution of the SSP problem, as discussed in Section 3.2.3: the G&T algorithm and the Hybrid G&T algorithm. These classes can be extended to add new constructive heuristics, such as NEH [24, 25].
- *TObserverGantt, GanttChartViewer*: These classes implement the Gantt chart visualizations.
- *TObserverEnergy, EnergyChartViewer*: These classes implement the electrical consumption chart visualizations.

In practice, these classes implement the components shown in Figure 12 as follows. Class *CDataReader* implements the Data reader component of the Data layer. The Job repository is implemented by the classes *CJobCollection* and *CJobTechnologicalRoute*, while the machine repository is implemented by classes *CMachineCollection* and *CMachineGroup*.

On the problem-specific layer, the problem representation is implemented by the *CGraphForSSP* class, which contains methods to implement the objective functions. The solution representation is implemented by the *CSchedule* class. Still on this layer, the framework has two ready-to-use heuristics, implemented by classes *CGTAlgorithmBase, CGTAlgorithmClassic, CGTAlgorithmHybrid*. New heuristics can be added to the framework by extending the *CGTAlgorithmBase* class.

Finally, the components of the visualization layer are implemented by four classes. Classes *TObserverGantt* and *GanttChartViewer* implement the Gantt chart component, while classes *TObserverEnergy* and *EnergyChartViewer* implement the electrical consumption chart component.

As discussed in Section 3 and shown in Figure 2 and Figure 12, the SSP-3M framework does not depend on the metaheuristic. In fact, we claim that it can be integrated with any metaheuristic. Section 4.2 shows the integration of SSP-3M with a third-party genetic algorithm library.

4 User experience and illustrative results

This section reports the user experience and some computational results obtained from applying the SSP-3M framework. The goal here is not necessarily to replicate the optimal solutions or numerical results competitive with the best specific, state-of-the-art implementations but, instead, to illustrate the versatility, extensibility, and independence of the optimization method of the framework.

Section 4.1 presents numerical results obtained with the framework implementations of the classic and hybrid G&T construction heuristics, showing that SSP-3M

provides practical implementations of the most fundamental tools. Section 4.2 illustrates the ease of integration of the SSP-3M framework with a third-party optimization library, the GAlib genetic algorithm library [37] (*independence of the optimization method*). Section 4.3 makes a case for the *versatility* of the framework, illustrating numerical results obtained with the integration of the SSP-3M framework with the GAlib genetic algorithm library [37] and a self-implemented GRASP heuristic [38] to solve classic instances of different SSP variants such as the flow shop, generalized flow shop, job shop, and flexible job shop. It also shows that the solutions obtained by the algorithms implemented with the framework are competitive with the best state-of-the-art algorithms. Section 4.4 discusses the ease of *extensibility* of the framework. Finally, Section 4.5 discusses managerial implications of the SSP-3M framework.

4.1 Construction heuristics: numerical results

Our first experimental evaluation aims at showing that the SSP-3M framework provides practical implementations of the most fundamental tools to solve SSP problems. It is possible to use the framework without extending it. For that, we built our experiment considering the classic and hybrid G&T algorithms and three well-known instances of different complexities proposed by Fisher and Thompson [33]: ft06 (low complexity), ft20 (medium complexity), and ft10 (high complexity). We performed 5000 runs of the classic G&T algorithm for each instance with random choices. We also combined the G&T algorithm with three different dispatching rules: shortest processing time (SPT), start as-early-as-possible (SEP), and earliest completion time (ECT), executing 5000 runs of each combination for each instance. Finally, we executed 5000 runs of the hybrid G&T algorithm using $\delta = 0.0$, 0.4, 0.7, and 1.0.

Detailed results for the three instances are available in [36]. For space's sake, we show only the results for the highest complexity instance, ft10. Figure 14 displays the histogram of the obtained makespan values over 5000 runs for each variant of the classic G&T algorithm. It illustrates the behavior of the different dispatching rules (SEP, SPT, and ECT) combined with the classic G&T algorithm to improve solution quality compared to the classic algorithm. It is interesting to notice that although line 10 of the G&T algorithm performs a randomized selection, its repetition over 5000 runs, combined with the dispatching rules SPT or ECT, always produces the same solution. This repetition is due to the nature of the input data (i.e., the processing times) since, at every iteration, only one operation satisfying the corresponding rule is ready to be selected. This situation further illustrates the richness of having different construction algorithms available combining distinct dispatching rules, which can be combined to construct a good schedule instead of being trapped in a bad solution.

Similarly, Figure 15 displays the histogram of the obtained makespan values over 5000 runs for each value of δ for the hybrid G&T algorithm. It indicates that the closer the value of δ is to zero, the better solutions the algorithm obtains.

The two construction heuristics and their variants obtained solutions close to the optimal values (indicated by the red lines in both figures, which correspond to the minimum makespan). Using different dispatching rules implemented in the framework

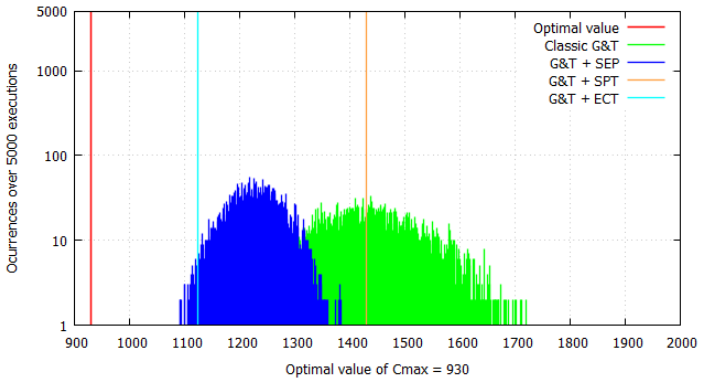


Fig. 14 Histogram of makespan values over 5000 runs of each variant of the classic G&T algorithm (random, SEP, SPT, and ECT) on instance ft10.

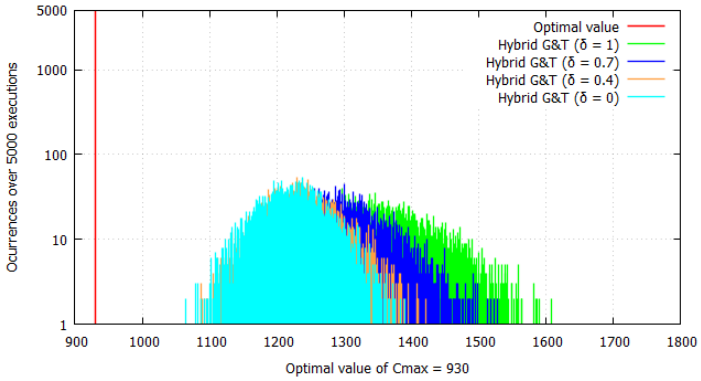


Fig. 15 Histogram of makespan values over 5000 runs of each variant of the hybrid G&T algorithm (δ =0.0, 0.4, 0.7, and 1.0) on instance ft10.

ensures a greater solution diversity, which might be helpful in the development of meta-heuristics such as genetic algorithms or GRASP. Both heuristics and their variants are available to the users of the framework.

4.2 Independence of the optimization library

The ease of integration of SSP-3M with higher-level optimization methods allows for the framework's independence from the applied optimization technique, which may be anyone.

This section illustrates this aspect by integrating SSP-3M with a third-party optimization library. With this goal, we used the GAlib library, a set of C++ genetic algorithm objects developed by the Massachusetts Institute of Technology. It

```
1  #include <ga/ga.h>
2  #include "ssp.model/CGraph.h"
3  #include "ssp.model/CDataReaderFile.h"
4  #include "ssp.model/CJobCollection.h"
5  #include "ssp.model/CMachineCollection.h"
6
7  CDataReader File("ft10.txt");                          // set data filename
8  CJobCollection JobsRepo;
9  CMachineCollection MachinesRepo;
10 float Objective(const GAGenome &);
11
12 int main(){
13     int nJobs, mMachines;
14     File.ReadInstanceSize(&nJobs, &mMachines);         // read instance size
15     File.ReadMachineData(&CollectionM);                // read machine data
16     File.ReadPieceData(&CollectionP, CollectionM);     // read job data
17
18     GA1DArrayGenome genome(nJobs * mMachines, Objective);  // create the chromosome
19     GASimpleGA ga(genome);                             // create the GA instance
20     ga.populationSize(150);                            // set population size
21     ga.nGenerations(500);                              // set generation number
22     ga.pMutation(0.01);                                // set mutation factor
23     ga.pCrossover(0.85);                               // set crossover factor
24     ga.evolve();                                       // evolve the GA
25     cout << ga.statistics() << endl;                   // print out the results
26     return 0;
27 }
28
29 float Objective (const GAGenome &genome)
30 {
31     CGraphForSSP Graph;
32
33     if (isValidGenome(genome))
34     {
35         JobsRepo.CreateGraph_NxM_Dimension(&Graph, MachinesRepo, genome);
36         return -Graph.Makespan();
37     }
38     return 0.0f;
39 }
```

Fig. 16 C++ code snippet showing the integration of the SSP-3M framework with the GAlib to solve an instance of the job shop SSP variant.

includes tools for using genetic algorithms in any C++ program implementing several representations and genetic operators [37].

Figure 16 displays the C++ code snippet showing the integration of the SSP-3M framework with the GALib to solve an instance of the job shop SSP variant. The lines highlighted in blue correspond to the source code of the SSP-3M framework, while the lines highlighted in yellow correspond to the GAlib resources. Lines not highlighted correspond to the application and the integration points between SSP-3M and GAlib.

Lines 1 to 5 contain the header files, including the GAlib library and the SSP-3M framework. The SSP-3M variables are declared in lines 7 to 10. Lines 12 to 27 correspond to the application that executes the genetic algorithm (GA) integrated with the framework SSP-3M inside the main function. Line 13 declares the variables that store the number of jobs and the number of machines. Lines 14 to 16 read the input data: number of jobs and machines, machine data, and job data. These lines use the classes defined in the data layer to read the data from a file and store the information in memory. Line 18 defines the chromosome to be used by the genetic

algorithm. The size of the chromosome and a reference to the objective function used in evaluating the solution quality are passed as parameters. Line 19 creates an instance of the class GASimpleGA, which contains the genetic algorithm. Lines 20 to 23 define the parameters of the GA. Line 24 executes the evolution of the genetic algorithm over the specified number of generations. Line 25 outputs the results. Line 26 concludes the execution and returns the status code of success to the operating system.

Lines 29 to 39 implement the objective function that is passed as a reference to the genetic algorithm in line 18. Line 31 declares the instance of the class graph that implements the data structure, the graph traversal, and all objective function calculations. Line 33 validates the solution passed as a parameter. If the solution is valid, then line 35 creates the directed disjunctive graph (illustrated in Figure 5). Line 36 executes the traversal of the direct disjunctive graph. It returns the makespan multiplied by -1 since the GAlib handles a maximization problem, while we are interested in the minimization of the makespan. If the solution passed as a parameter in line 33 was not validated in line 33, then line 38 returns a dummy (zero) makespan value.

The code snippet in Figure 16 illustrates the simplicity of the integration of the SSP-3M framework with a third-party optimization library or other external optimization routines, either exact (such as mixed linear integer programming) or approximated (such as metaheuristics). In addition, the modularity and the encapsulation of the main framework's functions make the resulting code clean and easy to read and maintain.

4.3 Versatility: solving different problem variants

In order to illustrate the versatility of the SSP-3M framework, this section presents the results of its integration with two metaheuristics: the GAlib genetic algorithm library and a self-implemented GRASP heuristic. The results reported in the following correspond to their application to instances of the permutation flow shop, generalized flow shop, job shop, and flexible job shop SSP variants. Each instance was executed with each metaheuristic integrated with the SSP-3M framework.

4.3.1 Experimental setting: metaheuristics

Since our goal here was not to obtain the best possible solutions but to show that integrating the SSP-3M framework with existing metaheuristics is feasible, we neither sought very sophisticated implementations of the metaheuristics nor calibrated them to find the best parameter settings. Instead, the parameters used to execute the genetic algorithm and the GRASP were obtained empirically after some preliminary computational experiments with a few training instances.

The integration with the GAlib [37] was described in Section 4.2 and used the typical settings. The number of generations was fixed at 1000, the population size was set to 200, the crossover probability was set to 0.85, the mutation factor was fixed at 0.01, and we used selection by ranking and the PMX crossover operator to guarantee feasible offspring.

Our implementation of a GRASP heuristic is made up of the typical building blocks: the construction phase, the local search phase, and a path-relinking

procedure [38]. The construction phase is implemented with a greedy randomized construction heuristic, for which we made use of the randomization of the classic G&T algorithm (Algorithm 1 in the Appendix) hybridized with the SEP (start as-early-as-possible) dispatching rule for randomization (we select among the best candidates using $\alpha = 0.6$ to create the restricted candidate list). The local search follows a first improving strategy using a swap move in which any two operations of the vector representing the operation sequence may be exchanged. Path-relinking [39–41] is implemented following a back-and-forward strategy, with the size of the elite set fixed to 15. The heuristic stops after 1000 iterations.

4.3.2 Computational results on different problem variants

The computational experiments that are the object of this section involved a total of 364 benchmark problems: 120 permutation flow shop instances and two generalized flow shop instances available from Beasley [42], 122 job shop instances available from Beasley [43], and 120 flexible job shop instances available from Mastrolilli [44]. Detailed results for the test problems are available in [36]. For the sake of space, we only show the results obtained for some instances of each shop scheduling problem variant.

Tables 2 to 6 below are organized as follows. The first column of each table shows the instance name. The second and third columns indicate the number of jobs and machines. The fourth column gives a lower bound (LB) whenever available. The fifth column gives an upper bound to the makespan. The sixth and seventh columns show the results obtained with the two metaheuristics integrated with the SSP-3M framework. Values in boldface correspond to solutions that improved or matched the reported upper bound. The instances we use do not have job recirculation, so they each have at most m operations, where m is the number of machines shown in Tables 2 to 6.

Table 2 presents the results obtained for ten instances of the permutation flow shop. The lower bound (LB) was derived by Taillard [45]. The upper bound (Best) is the value of the best solution value reported in Taillard [45].

Table 2 Makespan results for ten instances of the permutation flow shop problem.

Instance	Jobs	Machines	Bounds		SSP-3M	SSP-3M
	n	m	LB	Best	+GA	+GRASP
ta001	20	5	1232	1278	**1278**	**1278**
ta002	20	5	1290	1359	**1359**	1366
ta003	20	5	1073	1081	**1081**	1088
ta004	20	5	1268	1293	**1293**	1296
ta005	20	5	1198	1235	**1235**	1244
ta006	20	5	1180	1195	**1195**	1203
ta007	20	5	1226	1226	1239	1239
ta008	20	5	1170	1206	**1206**	1222
ta009	20	5	1206	1230	**1230**	1237
ta010	20	5	1082	1108	**1108**	1117

Table 3 shows the results for the two instances of the generalized flow shop problem proposed by Heller [46]. The best-known solution values (Best) were obtained by the adaptive-learning approach of Agarwal et al. [47].

Table 3 Makespan results for two instances of the generalized flow shop problem.

Instance	Jobs	Machines	Bounds		SSP-3M	SSP-3M
	n	m	LB	Best	+GA	+GRASP
hel01	100	10	—	516	518	524
hel02	20	10	—	136	139	141

Table 4 displays the results for 15 instances of the job shop problem proposed by Lawrence [35]. The upper bounds displayed in the fifth column (Best) are the best-known solutions at the time of writing, see e.g. [48]. We notice that both heuristics matched the best-known solutions for all instances in this table.

Table 5 shows the results for ten hard job shop instances [48–50] proposed by Adams et al. [51]. The lower bounds (LB) and the best-known solutions (Best) were obtained by Applegate and Cook [48].

Table 4 Makespan results for fifteen instances of the job shop problem.

Instance	Jobs	Machines	Bounds		SSP-3M	SSP-3M
	n	m	LB	Best	+GA	+GRASP
la01	10	5	—	666	**666**	**666**
la02	10	5	—	655	**655**	**655**
la03	10	5	—	597	**597**	**597**
la04	10	5	—	590	**590**	**590**
la05	10	5	—	593	**593**	**593**
la06	15	5	—	926	**926**	**926**
la07	15	5	—	890	**890**	**890**
la08	15	5	—	863	**863**	**863**
la09	15	5	—	951	**951**	**951**
la10	15	5	—	958	**958**	**958**
la11	20	5	—	1222	**1222**	**1222**
la12	20	5	—	1039	**1039**	**1039**
la13	20	5	—	1150	**1150**	**1150**
la14	20	5	—	1292	**1292**	**1292**
la15	20	5	—	1207	**1207**	**1207**

Finally, Table 6 presents the results obtained for ten instances of the flexible job shop problem on parallel machines proposed by Brandimarte [52]. The lower bounds were obtained from Brandimarte [52] while the best-known solution values (Best) were extracted from Bagheri et al. [53] and Behnke and Geiger [54].

More important than the good quality of the above numerical results themselves is that they show that the SSP-3M framework can be successfully integrated

Table 5 Makespan results for ten high-difficulty instances of the job-shop problem.

Instance	Jobs	Machines	Bounds		SSP-3M	SSP-3M
	n	m	LB	Best	+GA	+GRASP
abz7	20	15	654	655	683	714
abz8	20	15	635	675	694	727
abz9	20	15	656	686	712	751
la21	15	10	1046	1046	1050	1050
la24	15	10	935	935	944	950
la25	15	10	977	977	978	984
la27	20	10	1235	1235	1246	1252
la29	20	10	1120	1154	1176	1193
la38	15	15	1184	1198	1225	1274
la40	15	15	1222	1228	1241	1282

with different optimization methods for different problem variants. We showed that the framework works well with two different metaheuristics of distinct natures: a population-based genetic algorithm and a trajectory-based GRASP. The solutions obtained with the straightforward integration of these two metaheuristics with the framework are competitive with the best-known solutions obtained by state-of-the-art methods even without a fine-tuning of their parameters.

Table 6 Makespan results for ten instances of the flexible job shop problem on parallel machines.

Instance	Jobs	Machines	Bounds		SSP-3M	SSP-3M
	n	m	LB	Best	+GA	+GRASP
mk01	10	6	36	39	40	40
mk02	10	6	24	26	27	27
mk03	15	8	204	204	**204**	**204**
mk04	15	8	48	60	61	63
mk05	15	4	168	173	**173**	176
mk06	10	15	33	59	64	65
mk07	20	5	133	139	**139**	140
mk08	20	10	523	523	**523**	**523**
mk09	20	10	299	307	**307**	**307**
mk10	20	15	165	212	218	221

4.4 Ease of extensibility

Figure 2 shows three different extension points to the framework: data input, objective function, and heuristics. In this section, we discuss how the framework can be extended to add new forms of data input, new objective functions, and new heuristics.

To prepare the framework to read new types of input files, the developer must extend the *CDataReader* class. Currently, the framework provides an implementation to read text files like the one shown in Figure 3. Assuming the developer wants to

allow JSON input files, all that has to be done is to extend the *CDataReader* class and implement the methods *readInstanceSize()*, *readMachineData()*, *readJobData()*, and *readWorkcenterData()* shown in Figure 13. Similar changes could be made to allow the framework to read from a database. In this case, in addition to implementing the aforementioned methods, the developer would need to add new attributes to represent the database connection details, such as the connection string, username, and password. It would also be necessary to add new methods to connect and disconnect to/from the database.

To include a new objective function, the developer needs to extend the *CGraphForSSP* class. This class provides several ready-to-use objective functions, such as *makespan()* and *energyConsumption()*, among others, and the developer can add new ones as needed. Finally, to add a new heuristic into the framework, the developer must extend the *TGTAlgoritmBase* class. Again, the framework provides two ready-to-use constructive heuristic, but others can be added as needed.

As for adding new graphical representations, the extension is a bit more trickery, but it is totally possible. First, the developer creates an abstract class to represent the new type of graphical representation (for example TNewGraphic). This abstract class should have the declaration of a method to create the new graph. Then, it is necessary to create a concrete class that implements the new graphical representation (for example, CNewGraphic), and that inherits from the abstract class TNewGraphic. The concrete class will then implement the generation of the new type of graph (necessary data can be passed as parameters). Then, the Observer pattern can be used to pass a reference of the new concrete class (CNewGraphic, in this example) to the CGraphForSSP class.

4.5 Managerial implications

Deploying a manufacturing scheduling system in a company requires a great effort in the design and development phases of the system, where several important decisions should be made. One of these decisions comprises choosing existing software components that could fulfill the company's requirements and reduce the development time.

The SSP-3M framework is a good choice due to several reasons. It is an open-source, ready-to-use software component. It can be applied to multiple production scenarios since it solves several variants of the shop scheduling problem. The most commonly used objective functions are implemented in the framework, but others can be easily added by the user if needed. Additionally, since the framework does not depend on the optimization method, it can be integrated with any metaheuristic (or any other optimization strategy) chosen by the company. Thus, using SSP-3M reduces cost (since it is free of charge) and development time (since it provides several ready-to-use functionalities).

Addressing practical day-to-day issues of the industry, such as the integration of the scheduling with Computer-aided Process Planning (CAPP), Advanced Planning and Scheduling (ASP), Integrated Process Planning and Scheduling (IPPS) and Computer Integrated Manufacturing (CIM) were shown to be important [3]. This integration is only possible when the scheduling solution is versatile and able to handle different SSP

variants. In fact, this versatility needs to be improved in the literature, and the SSP-3M framework is the only open-source, ready-to-use, and fully implemented software solution able to cover this gap successfully.

Also, considering characteristics of real-life production environments, such as machine flexibility, recirculation of jobs, task priorities, due dates, and energy consumption, the SSP-3M framework contributes to reducing the gap between theory and practice. The quality of the numerical results reported in Section 4.3 shows that the framework can be integrated successfully with different optimization techniques.

5 Concluding remarks

We contribute by designing and implementing SSP-3M, an open-source framework for shop scheduling problems. It is the only open-source solution available in the literature. The framework is designed to be versatile, extensible, and independent of the optimization method.

On the *versatility* side, it deals with the six existing variants of the SSP: the job shop, flow shop, permutation flow shop, generalized flow shop, flexible flow shop, and flexible job shop problems. The framework can support other problem variants as long as a directed disjunctive graph can represent them. To the best of our knowledge, the SSP-3M framework is the first in the literature to handle more than one SSP variant (see Table 1).

Our experimental evaluation makes a case for the versatility of the framework by illustrating numerical results obtained with the integration of the SSP-3M framework with the GAlib genetic algorithm library and a self-implemented GRASP heuristic to solve classic instances of different SSP variants, such as the flow shop, generalized flow shop, job shop, and flexible job shop. It also shows that the solutions obtained by the algorithms implemented with the framework are competitive with those obtained by the best state-of-the-art algorithms.

As for the *extensibility*, the user can add new forms of input data, new objective functions, and new heuristics. All of these can be easily done by extending the framework's classes, which are implemented in C++. The use of C++ is due to two reasons: it is object-oriented and thus ready to provide easy-to-extend capabilities, and it is fast in terms of execution time, a crucial feature when solving challenging optimization problems such as the SSP. The framework's implementation in C++ is not a limitation, since the ideas behind the framework are generic and can be implemented in other object-oriented programming languages.

Finally, the *independence of the optimization method* was illustrated in Section 4.2, where we showed the integration of SSP-3M with a third-party genetic algorithm.

The SSP-3M framework is thus a good choice to be deployed as a manufacturing scheduling system. It is an open-source, ready-to-use software component. It can be applied to multiple production scenarios since it solves several variants of the scheduling problem.

As future work, we intend to extend the framework to include other restrictions such as carrier times between machines; setup and configuration times of the

machines; machine availability (machines may become unavailable due to maintenance or machine breakdown); buffers; and release times.

Competing interests

The authors have no competing interests to declare that are relevant to the content of this article.

Software availability statement

The open-source, ready-to-use framework SSP-3M coded in C++ is available at the project repository at https://github.com/cherreram2012/jssp-framework [36].

Data sharing

The data that support the findings of this study are available from repositories at http://people.brunel.ac.uk/~mastjjb/jeb/orlib/flowshopinfo.html [42], http://people.brunel.ac.uk/~mastjjb/jeb/orlib/jobshopinfo.html [43], and https://people.idsia.ch/~monaldo/fjsp.html [44]. Full numerical results are available at the project repository at https://github.com/cherreram2012/jssp-framework/wiki/Benchmark-results [55].

Acknowledgments

The work of Celso C. Ribeiro was supported by research grants CNPq 309869/2020-0 and FAPERJ E-26/200.926/2021. Vanessa Braganholo was supported by research grant CNPq 305020/2019-6.

Appendix A Greedy randomized G&T algorithm

The greedy randomized G&T algorithm of Giffler and Thompson [31] takes as input the set of jobs $J = \{J_1, \ldots, J_n\}$, the set of machines $M = \{M_1, \ldots, M_m\}$, and, for each job J_i, $i = 1, \ldots, n$, the number of operations $\#op(i)$ to be processed, a list $L^i = [L^i[1], \ldots, L^i[\#op(i)]]$ with the sequence of machines where it has to be processed, and the processing times $p_{i,1}, \ldots, p_{i,\#op(i)}$ of each operation. The algorithm outputs a schedule represented by an operation sequence (possibly with repetitions), with each operation defined by a job, a machine, and the time this job should start to be processed in this machine. The main steps of its pseudo-code appear in Algorithm 1, which is our formalization of the original algorithm textually described in [31].

Line 1 initializes the set A that stores the candidate operations, i.e., those that are ready to be scheduled. Each iteration $i = 1, \ldots, n$ of the loop in lines 2 to 6 places the first operation of job J_i in the candidate set A in line 3 and sets its starting time to 0 in line 4. Line 5 sets to 1 the position of the first operation of job J_i ready to be scheduled. Line 7 sets to 0 the counter of scheduled operations. The while loop in lines 8 to 20 runs until all operations are scheduled. Line 9 selects from the candidate set A the operation $o_{i',k'}$ with the shortest completion time, corresponding to the execution of job $J_{i'}$ in machine $M_{k'}$. Line 10 creates the set $B \subseteq A$ formed by the candidate operations that guarantee the construction of an active schedule. It contains all operations that use machine $M_{k'}$ and may start before the shortest completion time determined in line 9. Line 11 randomly selects an operation o_{i^*,k^*}

Algorithm 1 G&T algorithm

Require: $J = \{J_1, \ldots, J_n\}$; $M = \{M_1, \ldots, M_m\}$; $L^i = [L^i[1], \ldots, L^i[\#op(i)]]$: $i = 1, \ldots, n$; $p_{ij} : i = 1, \ldots, n; j = 1, \ldots, \#op(i)$
Ensure: *schedule*[list]
1: $A \leftarrow \emptyset$
2: **for** $i = i, \ldots, n$ **do**
3: $A \leftarrow A \cup \{o_{i,L^i[1]}\}$
4: $s_{i,L^i[1]} \leftarrow 0$
5: $position_i \leftarrow 1$
6: **end for**
7: Set $counter \leftarrow 0$
8: **while** $(A \neq \emptyset)$ **do**
9: Set $o_{i',k'} = \text{argmin}_{o_{ik} \in A}\{s_{ik} + p_{ik}\}$
10: $B \leftarrow \{o_{ik} \in A : k = k' \text{ and } s_{ik} < s_{i',k'} + p_{i',k'}\}$
11: Randomly select $o_{i^*,k^*} \in B$
12: $A \leftarrow A \setminus \{o_{i^*,k^*}\}$
13: $counter \leftarrow counter + 1$
14: $schedule[counter] \leftarrow o_{i^*,k^*}$
15: **if** $position_{i^*} < \#op(i^*)$ **then**
16: $position_{i^*} \leftarrow position_{i^*} + 1$
17: $A \leftarrow A \cup \{o_{i^*,L^{i^*}[position_{i^*}]}\}$
18: $s_{i^*,L^{i^*}[position_{i^*}]} \leftarrow s_{i^*,k^*} + p_{i^*,k^*}$
19: **end if**
20: **end while**
21: **return** *schedule*

from B. Line 12 removes the newly selected operation from the candidate set A. Line 13 updates the counter of scheduled operations, and line 14 schedules operation o_{i^*,k^*} in this position of the operation sequence. Line 15 checks if operation o_{i^*,k^*} is not the last of job J_{i^*}. In this case, line 16 updates the position of the subsequent operation of job J_{i^*} ready to be scheduled. Line 17 adds the next operation of job J_i to the candidate set A, and its starting time is set to the completion time of operation o_{i^*,k^*} in line 18. Line 21 returns the schedule represented as an operation sequence.

We remark that other dispatching rules can be used at step 11 instead of a simple random choice, such as shortest processing time (SPT), longest processing time (LPT), start as-early-as-possible (SEP), earliest completion time (ECT), earliest due date (EDD), longest number of successors (LNS), and shortest number of successors (SNS). They all lead to active schedules and different variants of the algorithm [56].

References

[1] Brucker, P.: Scheduling Algorithms, 5th edn. Springer, Berlin (2007)

[2] Bahouth, S.: Taxonomy of shop scheduling. Online reference available at SSRN: http://dx.doi.org/10.2139/ssrn.2447152, last access on January 8, 2023 (2014)

[3] Márquez, C.R.H., Ribeiro, C.C.: Shop scheduling in manufacturing environments: A review. International Transactions in Operational Research **29**, 3237–3293

(2022)

[4] Pinedo, M.L.: Scheduling: Theory, Algorithms and Systems. Springer, New York (2012)

[5] Salido, M.A., Escamilla, J., Barber, F., Giret, A.: Rescheduling in job-shop problems for sustainable manufacturing systems. Journal of Cleaner Production **162**, 121–132 (2017)

[6] King, J.R.: The theory-practice gap in job-shop scheduling. The Production Engineer **55**, 137–143 (1976)

[7] McKay, K.N., Buzacott, J.A.: Application of computerized production control systems in job shop environments. Computers in Industry **42**, 79–97 (2000)

[8] Arisha, A., Young, P.: Job shop scheduling problem: An overview. In: International Conference for Flexible Automation and Intelligent Manufacturing, pp. 682–693. Technological University Dublin, Dublin (2001)

[9] Buxey, G.: Production scheduling: Practice and theory. European Journal of Operational Research **39**, 17–31 (1989)

[10] Reisman, A., Kumar, A., Motwani, J.: Flowshop scheduling/sequencing research: A statistical review of the literature, 1952–1994. IEEE Transactions on Engineering Management **44**, 316–329 (1997)

[11] Pongcharoen, P., Hicks, C., Braiden, P.M.: The development of genetic algorithms for the finite capacity scheduling of complex products, with multiple levels of product structure. European Journal of Operational Research **152**, 215–225 (2004)

[12] Framinan, J.M., Ruiz, R.: Guidelines for the deployment and implementation of manufacturing scheduling systems. International Journal of Production Research **50**, 1799–1812 (2012)

[13] Sun, D., Lin, L.: A dynamic job shop scheduling framework: A backward approach. International Journal of Production Research **32**, 967–985 (1994)

[14] Lee, C.-Y., Piramuthu, S., Tsai, Y.-K.: Job shop scheduling with a genetic algorithm and machine learning. International Journal of Production Research **35**, 1171–1191 (1997)

[15] Vieira, G.E., Herrmann, J.W., Lin, E.: Rescheduling manufacturing systems: A framework of strategies, policies, and methods. Journal of Scheduling **6**, 39–62 (2003)

[16] Lau, H.Y.K., Qiu, X.: An artificial immune systems (AIS)-based unified framework for general job shop scheduling. IFAC Proceedings Volumes **47**, 6186–6191

(2014)

[17] Fumagalli, L., Macchi, M., Negri, E., Polenghi, A., Sottoriva, E.: Simulation-supported framework for job shop scheduling with genetic algorithm. In: Proceedings of the Summer School Francesco Turco, vol. 2017-Septe, pp. 271–278 (2017)

[18] Liu, W., Ma, W., Hu, Y., Jin, M., Li, K., Chang, X., Yu, X.: Production planning for stochastic manufacturing/remanufacturing system with demand substitution using a hybrid ant colony system algorithm. Journal of Cleaner Production **213**, 999–1010 (2019)

[19] Yen, B.P.C., Pinedo, M.: On the design and development of scheduling systems. In: Proceedings of the Fourth International Conference on Computer Integrated Manufacturing and Automation Technology, pp. 197–204. IEEE Computer Society Press, Troy (1994)

[20] Pinedo, M., Yen, B.P.C.: On the design and development of object-oriented scheduling systems. Annals of Operations Research **70**, 359–378 (1997)

[21] Bidot, J., Vidal, T., Laborie, P., Beck, J.C.: A theoretic and practical framework for scheduling in a stochastic environment. Journal of Scheduling **12**, 315–344 (2009)

[22] Framinan, J.M., Ruiz, R.: Architecture of manufacturing scheduling systems: Literature review and an integrated proposal. European Journal of Operational Research **205**, 237–246 (2010)

[23] Romero-Silva, R., Santos, J., Hurtado, M.: A framework for studying practical production scheduling. Production Planning & Control **26**, 438–450 (2015)

[24] Liu, W., Jin, Y., Price, M.: A new improved NEH heuristic for permutation flowshop scheduling problems. International Journal of Production Economics **193**, 21–30 (2017)

[25] Nawaz, M., Enscore, E.E., Ham, I.: A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. Omega **11**, 91–95 (1983)

[26] Brucker, P., Jurisch, B., Sievers, B.: A branch and bound algorithm for the job-shop scheduling problem. Discrete Applied Mathematics **49**, 107–127 (1994)

[27] Roy, B., Sussmann, B.: Les problemes d'ordonnancement avec contraintes disjonctives. Technical report, SEMA, Paris (1964)

[28] Holsapple, C.W., Jacob, V.S., Pakath, R., Zaveri, J.S.: A genetics-based hybrid scheduler for generating static schedules in flexible manufacturing contexts. IEEE Transactions on Systems, Man, and Cybernetics **23**, 953–972 (1993)

[29] Fang, H.-l., Ross, P., Corne, D.: A promising genetic algorithm approach to job-shop scheduling, rescheduling, and open-shop scheduling problems. In: Forrest, S. (ed.) Proceedings of the Fifth International Conference on Genetic Algorithms, pp. 375–382. Morgan Kaufmann, San Francisco (1993)

[30] Bierwirth, C.: A generalized permutation approach to job shop scheduling with genetic algorithms. OR Spektrum **17**, 87–92 (1995)

[31] Giffler, B., Thompson, G.L.: Algorithms for solving production-scheduling problems. Operations Research **8**, 487–503 (1960)

[32] Bierwirth, C., Mattfeld, D.C.: Production scheduling and rescheduling with genetic algorithms. Evolutionary Computation **7**, 1–17 (1999)

[33] Fisher, H., Thompson, G.L.: Probabilistic learning combinations of local job shop scheduling rules. In: Muth, J.F., Thompson, G.L. (eds.) Industrial Scheduling, pp. 225–251. Prentice Hall, Englewood Cliffs (1963)

[34] Blażewicz, J., Ecker, K., Pesch, E., Schmidt, G., Węglarz, J.: Handbook on Scheduling: From Theory to Applications. Springer, Berlin (2007)

[35] Lawrence, S.: Resource constrained project scheduling: An experimental investigation of heuristic scheduling techniques (Supplement). Technical report, Carnegie-Mellon University, Pittsburgh (1984)

[36] Márquez, C.R.H.: SSP-3M framework repository. Online reference at https://github.com/cherreram2012/jssp-framework, last access on June 5, 2023 (2023)

[37] Wall, M.: GAlib: A C++ Library of Genetic Algorithm Components. Online reference at http://lancet.mit.edu/ga/, last access on May 7, 2023 (1996)

[38] Resende, M.G.C., Ribeiro, C.C.: Optimization by GRASP: Greedy Randomized Adaptive Search Procedures. Springer, New York (2016)

[39] Resende, M.G.C., Ribeiro, C.C.: GRASP with path-relinking: Recent advances and applications. In: Ibaraki, T., Nonobe, K., Yagiura, M. (eds.) Metaheuristics: Progress as Real Problem Solvers, pp. 29–63. Springer, New York (2005)

[40] Resende, M.G.C., Ribeiro, C.C., Glover, F., Martí, R.: Scatter search and path-relinking: Fundamentals, advances, and applications. In: Gendreau, M., Potvin, J.-Y. (eds.) Handbook of Metaheuristics, 2nd edn., pp. 87–107. Springer, New York (2010)

[41] Ribeiro, C.C., Resende, M.G.C.: Path-relinking intensification methods for stochastic local search algorithms. Journal of Heuristics **18**, 193–214 (2012)

[42] Beasley, J.E.: OR-Library: Flow shop scheduling. Online reference at http://people.brunel.ac.uk/~mastjjb/jeb/orlib/flowshopinfo.html, last access on June 8,

2023 (2023)

[43] Beasley, J.E.: OR-Library: Job shop scheduling. Online reference at http://people.brunel.ac.uk/~mastjjb/jeb/orlib/jobshopinfo.html, last access om June 8, 2023 (2023)

[44] Mastrolilli, M.: Flexible Job Shop Problem. Online reference at https://people.idsia.ch/~monaldo/fjsp.html, last access on June 8, 2023 (2023)

[45] Taillard, E.: Benchmarks for basic scheduling problems. European Journal of Operational Research **64**, 278–285 (1993)

[46] Heller, J.: Some numerical experiments for an $M \times J$ flow shop and its decision-theoretical aspects. Operations Research **8**, 178–184 (1960)

[47] Agarwal, A., Colak, S., Eryarsoy, E.: Improvement heuristic for the flow-shop scheduling problem: An adaptive-learning approach. European Journal of Operational Research **169**, 801–815 (2006)

[48] Applegate, D., Cook, W.: A computational study of the job-shop scheduling problem. ORSA Journal on Computing **3**, 149–156 (1991)

[49] Yamada, T.: Studies on metaheuristics for jobshop and flowshop scheduling problems. PhD thesis, Kyoto University (2003)

[50] Xia, W.-J., Wu, Z.-M.: A hybrid particle swarm optimization approach for the job-shop scheduling problem. The International Journal of Advanced Manufacturing Technology **29**, 360–366 (2006)

[51] Adams, J., Balas, E., Zawack, D.: The shifting bottleneck procedure for job shop scheduling. Management Science **34**, 391–401 (1988)

[52] Brandimarte, P.: Routing and scheduling in a flexible job shop by tabu search. Annals of Operations Research **41**, 157–183 (1993)

[53] Bagheri, A., Zandieh, M., Mahdavi, I., Yazdani, M.: An artificial immune algorithm for the flexible job-shop scheduling problem. Future Generation Computer Systems **26**, 533–541 (2010)

[54] Behnke, D., Geiger, M.J.: Test instances for the flexible job shop scheduling problem with work centers. Technical Report RR-12-01-01, Helmut-Schmidt Universitat, Hamburg (2012)

[55] Márquez, C.R.H.: Benchmark results for the SSP-3M framework. Online reference at https://github.com/cherreram2012/jssp-framework/wiki/Benchmark-results, last access on June 5, 2023 (2023)

[56] Pfeiffer, A., Kádár, B., Monostori, L.: Stability-oriented evaluation of rescheduling

strategies, by using simulation. Computers in Industry **35**, 563–570 (2006)